THE SUMMIT PROJECTS

P3 CI/CD

Project 3 of 9 | Uses levels: Levels 3, 5 | ~10 hours

1 Goal and outcome

Make a `git push` the only way `shop-api` reaches production. A pull request runs lint and tests against real PostgreSQL and Redis; a merge to `main` builds the Project 2 image, scans it, pushes it to the GitHub Container Registry (GHCR) and deploys it to a server, with a smoke test after the deploy and an automatic rollback if the smoke test fails.

The pipeline concepts are covered in depth in Level 5 Capstone 5 (which builds a blue/green pipeline) and the repository mechanics in the Level 3 capstone Capstone 3. This project is the **simplest pipeline that is still honest**: one repository, one workflow, one environment, and a deploy you can roll back by re-running a tag.

You finish with:

- a GitHub repository `shop-api` with `main` protected: changes arrive only through pull requests with a green `ci` check;
- a workflow with three jobs: `test` (services: PostgreSQL, Redis), `image` (build, scan, push) and `deploy` (SSH to the server, gated by the `production` environment);
- images in GHCR tagged by commit (`sha-<full sha>`), never deployed by mutable tag;
- a `deploy.sh` on the server that pulls a specific tag, waits for health, runs `smoke.sh` and rolls back on failure;
- a **one-command rollback**: re-run the deploy with the previous tag.

2 Architecture

What changed from Project 2: the same Compose stack now runs on a **server** (re-use the Project 1 host with Docker installed, or any Docker-capable VM), the image is built by

GitHub's runners rather than your laptop, and a registry sits between build and run (NEW).

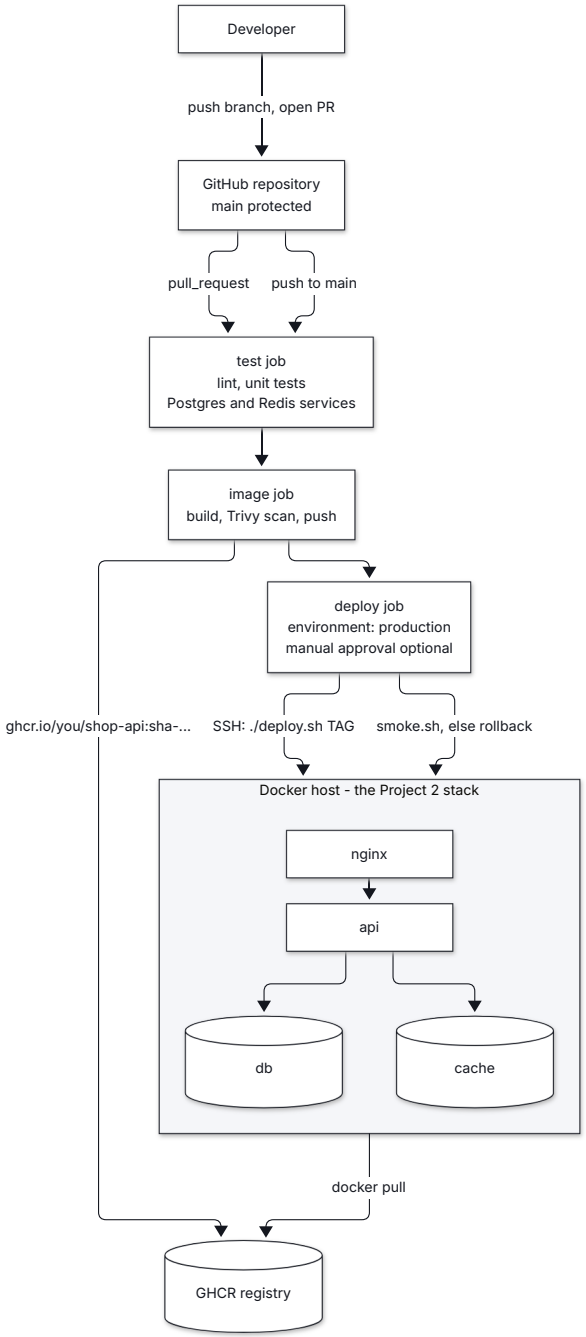


Figure 3-1

Deploy sequence on the server:

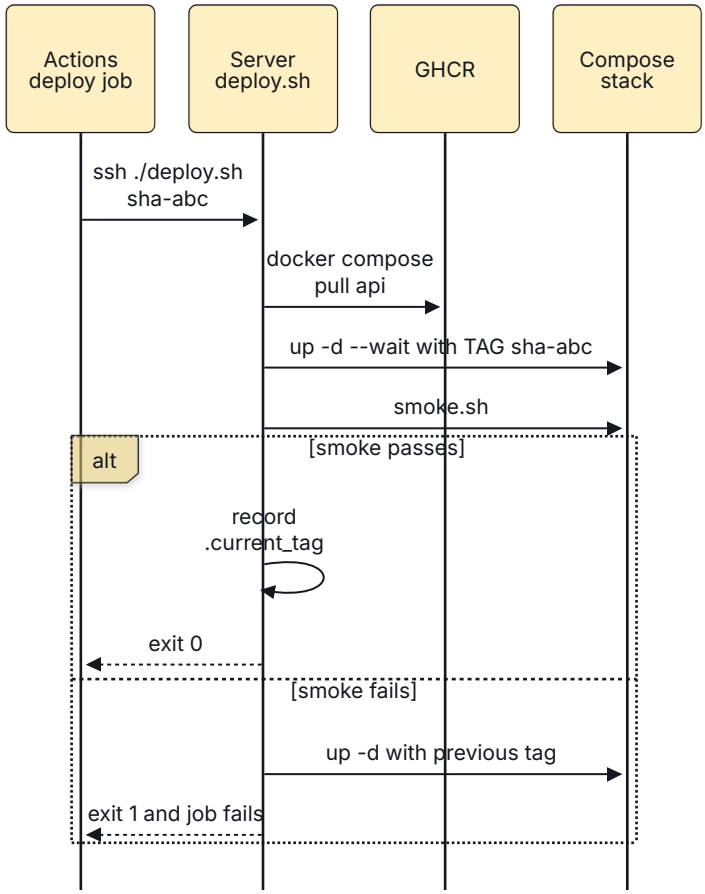


Figure 3-2

3 Prerequisites

- Everything from Project 2 working locally, and a GitHub account (free tier is enough for a public or private repository).
- A server that runs Docker: your Project 1 VPS (stop Nginx and the systemd service first so port 8080 and your data do not collide, or use a fresh VM). Cost: the VPS you already pay for.
- Git and GitHub: branching §3.1, tags §3.6, protected branches and pull requests §3.8, Actions workflows §3.11, environments and secrets §3.12.
- CI/CD: pipeline design §5.1, GitHub Actions pipelines §5.2, automated testing §5.3, Docker builds and registries §5.4, secrets §5.5, rollback §5.7.

- Tools: `git`, `gh` (GitHub CLI, optional), `ssh`.

4 Build it step by step

Milestone 1 — Repository and protected main (≈45 min)

```bash
cd shop && git init -b main && git add . && git commit -m "shop-api: containerized stack"
gh repo create shop-api --private --source=. --remote=origin --push      # or create it in the web UI
```

Under Settings, Branches (or Rulesets), protect `main`: require a pull request, require the status check `ci / test` and `ci / image`, block force pushes. Check that `.env` and `node_modules` are in `.gitignore` before the first push.

Tests need a script: add `"lint": "node --check server.js"` and `"test"` (for example `node --test`) to `package.json`, with at least one test that calls `/readyz` and `/products` against the CI database.

Milestone 2 — The workflow (≈2 h)

`.github/workflows/ci.yml` (syntax-checked as YAML; action versions are major tags, so in a real repository also pin them to commit SHAs and let Dependabot update them):

```yaml
name: ci

on:
  pull_request:
  push:
    branches: [main]

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-24.04
    services:
      postgres:
        image: postgres:17-alpine
        env:
          POSTGRES_USER: shop
```

```
          POSTGRES_PASSWORD: ci-only-not-a-secret
          POSTGRES_DB: shop_test
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U shop -d shop_test"
          --health-interval 5s --health-timeout 3s --health-retries 10
      redis:
        image: redis:7.4-alpine
        ports: ["6379:6379"]
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 5s --health-timeout 3s --health-retries 10
    env:
      DATABASE_URL: postgres://shop:ci-only-not-a-secret@localhost:5432/shop_test
      REDIS_URL: redis://localhost:6379
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: psql "$DATABASE_URL" -f db/init.sql
      - run: npm test

  image:
    needs: test
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - id: meta
        uses: docker/metadata-action@v5
        with:
          images: ghcr.io/${{ github.repository }}
          tags: |
            type=sha,format=long
      - name: Build (not pushed yet)
        uses: docker/build-push-action@v6
        with:
          context: .
          load: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - name: Scan before publishing
        uses: aquasecurity/trivy-action@0.28.0
        with:
```

```
          image-ref: ${{ fromJSON(steps.meta.outputs.json).tags[0] }}
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
      - name: Log in to GHCR
        if: github.event_name == 'push'
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - name: Push
        if: github.event_name == 'push'
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha

  deploy:
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    needs: image
    runs-on: ubuntu-24.04
    environment: production
    concurrency: deploy-production
    steps:
      - name: Configure SSH
        env:
          KEY: ${{ secrets.DEPLOY_SSH_KEY }}
          KNOWN_HOSTS: ${{ vars.DEPLOY_KNOWN_HOSTS }}
        run: |
          install -m 700 -d ~/.ssh
          printf '%s\n' "$KEY" > ~/.ssh/id_ed25519 && chmod 600 ~/.ssh/id_ed25519
          printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts
      - name: Deploy
        env:
          HOST: ${{ vars.DEPLOY_HOST }}
          TAG: sha-${{ github.sha }}
        run: ssh deploy@"$HOST" "/opt/shop/deploy.sh $TAG"
```

Two design points. First, the image is **scanned before it is pushed**, so a vulnerable image never reaches the registry from `main`. Second, `permissions:` is least privilege at the top and widened only where the job needs it (secrets and permissions §5.5).

Milestone 3 — Prepare the deploy target (≈1 h)

On the server: install Docker Engine, create `/opt/shop` containing `compose.yaml`, `nginx/`, `db/`, `smoke.sh`, a `.env` with `DB_PASS`, and this override `compose.deploy.yaml` (replace `you`):

```yaml
services:
  api:
    image: ghcr.io/you/shop-api:${TAG:?TAG is required}
    pull_policy: always
```

Create a deploy user with a dedicated key, and allow it to run Docker (membership in the `docker` group is root-equivalent, so restrict the key with `command=` in `authorized_keys` once you are comfortable). If the package is private, run `docker login ghcr.io` once with a fine-grained token that has only `read:packages`. In GitHub, create the `production` environment (add yourself as a required reviewer if you want a manual gate), then store the private key as the secret `DEPLOY_SSH_KEY` and the variables `DEPLOY_HOST` and `DEPLOY_KNOWN_HOSTS` (from `ssh-keyscan -t ed25519 <host>` checked against the host's real fingerprint).

`/opt/shop/deploy.sh`:

```bash
#!/usr/bin/env bash
set -euo pipefail
cd /opt/shop
NEW="${1:?usage: deploy.sh <tag>}"
PREV="$(cat .current_tag 2>/dev/null || echo none)"
dc() { TAG="$1" docker compose -f compose.yaml -f compose.deploy.yaml "${@:2}"; }

echo "deploying $NEW (previous: $PREV)"
dc "$NEW" pull api
dc "$NEW" up -d --no-build --wait --wait-timeout 90 || ok=no
if [ "${ok:-yes}" = yes ] && BASE=http://127.0.0.1:8080 ./smoke.sh; then
  echo "$NEW" > .current_tag
  [ "$PREV" ≠ none ] && echo "$PREV" > .previous_tag
  echo "deploy ok"
else
  echo "deploy FAILED" >&2
  if [ "$PREV" ≠ none ]; then
    echo "rolling back to $PREV" >&2
    dc "$PREV" up -d --no-build --wait --wait-timeout 90
  fi
  exit 1
fi
```

Milestone 4 — First pipeline run, then a change through a PR (≈1 h)

```bash
git switch -c feature/product-by-id
# edit server.js: add GET /products/:id, add a test
git commit -am "Add product by id" && git push -u origin HEAD
gh pr create --fill
gh pr checks --watch            # test and image pass; deploy does not run on PRs
gh pr merge --squash --delete-branch
gh run watch                    # test, image (push), deploy
curl -s http://<server>:8080/products/1
```

On the server `cat /opt/shop/.current_tag` should show the new `sha-...`; `docker compose ps` shows the new image digest.

Milestone 5 — Rollback (≈30 min)

Because every image is addressable by commit tag, rollback is a deploy of an older tag. Two ways:

```bash
ssh deploy@<server> '/opt/shop/deploy.sh "$(cat /opt/shop/.previous_tag)"'      # fastest, by hand
git revert -m 1 <merge-sha> && git push                                         # via Git, goes through CI again
```

Add a `workflow_dispatch` input `tag` and a manual `rollback` workflow (see the Level 5 roll-back topic §5.7) so anyone with access can do the first one from the Actions tab.

5 Tasks

- [ ] **T1 — Create the repository and protect `main`** *(≈30 min, beginner)*
  - **Goal:** nothing reaches `main` without review and a green check.
  - **Steps:**
    1. Push the project and configure the branch rule (PR required, status checks, no force-push).
    2. Try `git push origin main` directly from a clone.
  - **Done when:** the direct push is rejected with a protected-branch message.
- [ ] **T2 — Get `test` green on a PR** *(≈1 h, beginner)*
  - **Goal:** lint and tests run against real PostgreSQL and Redis service containers.
  - **Steps:**

1. Add `npm run lint`, `npm test` and a test hitting `/readyz` and `/products`.
2. Open a PR and watch `gh pr checks --watch`.
3. Introduce a failing test and confirm the PR cannot merge.

- **Done when:** a green PR can be merged, and a red PR is blocked by the required check.

☐ **T3 — Build, scan and publish an image** *(≈1 h, intermediate)*

- **Goal:** an immutable, traceable image in GHCR.
- **Steps:**
  1. Merge to `main` and open the package page for `shop-api`.
  2. Find the tag `sha-<full commit sha>` and the OCI labels linking back to the repository.
  3. Pull it locally: `docker pull ghcr.io/<you>/shop-api:sha-<sha>` and run `docker image inspect` to read its labels.
- **Done when:** the image exists for the merge commit, and you can trace it from the tag to the commit and the workflow run.

☐ **T4 — Deploy automatically with a gate** *(≈1.5 h, intermediate)*

- **Goal:** merge to `main` updates the running service.
- **Steps:**
  1. Configure the `production` environment, secrets and variables; add yourself as a required reviewer.
  2. Merge a visible change (add a field to `/readyz`), approve the deployment in the Actions UI.
  3. Check `curl -s http://<server>:8080/readyz` and `cat /opt/shop/.current_tag`.
- **Done when:** the new field is visible on the server within minutes of approving, and the deploy job log shows `deploy ok`.

☐ **T5 — Make the pipeline fail safely** *(≈1 h, intermediate)*

- **Goal:** a broken release does not stay deployed.
- **Steps:**
  1. Commit a change that makes `/readyz` return 500 only in production (for example a missing env var read at runtime), and merge it.
  2. Watch `deploy.sh` fail its smoke test and roll back.
  3. Read `docker compose ps` and the job log to explain which step failed.
- **Done when:** the deploy job is red, the previous tag is still serving traffic, and `.current_tag` was not updated.

- [ ] **T6 — Roll back by tag** *(≈30 min, intermediate)*
  - **Goal:** recover in under two minutes without touching Git.
  - **Steps:**
    1. Deploy a known-bad but passing change (a wrong price in the init data is fine).
    2. Run `deploy.sh` with the previous tag from `.previous_tag`.
    3. Time it with `time`.
  - **Done when:** the old behavior is back, and you have a measured rollback time written down.
- [ ] **T7 — Speed it up and harden it** *(≈1.5 h, advanced)*
  - **Goal:** a faster, less trusting pipeline.
  - **Steps:**
    1. Compare run times before and after enabling `cache: npm` and `type=gha` Docker cache.
    2. Pin every action to a commit SHA and add `dependabot.yml` for `github-actions` and `npm`.
    3. Add a Gitleaks or secret-scanning check and confirm a fake token in a PR is caught (secrets scanning §11.11).
  - **Done when:** warm-cache runs are measurably faster, all actions use SHAs, and a planted fake secret fails the build.
- [ ] **T8 — Prove it: ship a change end to end without the guide** *(≈1.5 h, advanced)*
  - **Goal:** demonstrate the whole flow and explain every trust relationship.
  - **Steps:**
    1. From a fresh clone, branch, change, PR, review (ask a friend or use a second account), merge, approve and verify on the server.
    2. Draw which credentials each job uses (`GITHUB_TOKEN`, SSH key, registry token) and what each can do if stolen.
    3. Explain why deploys use `sha-...` tags and never `latest`.
  - **Done when:** the change is live, you have the diagram, and your explanation names at least three ways the pipeline limits the damage of a compromised token.

6 Definition of done

- [ ] `main` is protected; the required checks are `ci / test` and `ci / image`.

- [ ] A PR shows lint, tests and a scan result; a merge produces a `sha—<commit>` image in GHCR.
- [ ] The deploy job runs only from `main`, uses the `production` environment, and authenticates with a dedicated key.
- [ ] No secret appears in the repository, workflow logs or image; CI-only credentials are obviously fake.
- [ ] A failing smoke test rolls back automatically; a manual rollback to a previous tag takes under two minutes.
- [ ] A new change takes a few minutes from merge to production without anyone logging in to the server.
- [ ] Workflow permissions are least privilege (`contents: read` by default).

7 Break it on purpose

1. **Poison the lockfile.** Change a dependency version in `package.json` without updating `package-lock.json`. `npm ci` fails in CI. Explain why `npm ci` is stricter than `npm install` and why that is good.
2. **Expire the deploy key.** Remove the public key from `authorized_keys` and merge a change. Read the failure in the deploy job. Then restore it. Note that the `image` job already published the image, so retry only the deploy.
3. **Race two deploys.** Merge two PRs quickly. The `concurrency` group serialises the deploys; remove it temporarily in a branch and observe how two `docker compose up` calls can collide. Restore it.
4. **Publish a vulnerable base.** Change the Dockerfile base to an old tag (for example `node:18-alpine`) in a PR. Trivy should block the merge. Decide, with the report open, when you would accept a finding and how to record that decision.

8 How this evolves into the next project

The pipeline still deploys onto **one hand-made server** whose IP, firewall rules and Docker install exist only in your memory and your notes. Project 4 — Terraform replaces that server with infrastructure defined as code on AWS: a VPC with public and private subnets, EC2 instances behind an Application Load Balancer, and a managed RDS PostgreSQL. The workflow you built here stays; only the deploy target changes, and the Terraform change itself will flow through its own pull-request pipeline.